UAS KOMUNIKASI DATA



Nama:

Nayunda Krisna Abi / 4524210077

Dosen Pengampu:

Dr. Ane Prasetyowati. ST., MT.

PROGRAM STUDI TEKNIK INFORMATIKA

UNIVERSITAS PANCASILA

2025

Kategori A: Pemahaman Konsep dan Protokol (Teoritis)

1. Perbandingan Protokol Kritis (Analisis)

Soal ini menguji pemahaman mendalam tentang alasan pemilihan TCP vs. UDP.

Soal: Jelaskan secara rinci tiga (3) perbedaan utama antara TCP dan UDP dari sudut pandang mekanisme Lapisan Transport. Berdasarkan perbedaan ini, jelaskan mengapa protokol TCP wajib digunakan untuk fitur Transfer File dalam proyek Anda, dan sebutkan konsekuensi teknis jika fitur tersebut dipaksakan menggunakan UDP.

JAWAB :

Tiga Perbedaan Utama:
1. Koneksi (Connection): TCP itu connection-oriented, ibaratnya harus "salaman" dulu (handshake) biar jalur siap sebelum kirim data. Kalau UDP itu connectionless, main kirim saja tanpa memastikan penerima siap atau tidak.
2. Keandalan (Reliability): TCP menjamin data sampai (reliable); kalau ada paket hilang di jalan, pasti dikirim ulang. UDP sifatnya best-effort (sebisanya), kalau hilang ya sudah, tidak ada garansi pengiriman ulang.
3. Urutan (Sequencing): TCP menjamin paket disusun ulang sesuai nomor urut, jadi data tidak akan acak-acakan. UDP tidak menjamin urutan; paket yang dikirim pertama bisa saja sampainya belakangan.

Kenapa Transfer File Wajib TCP? Karena file (seperti .docx atau .exe) itu sangat sensitif terhadap data. Hilang satu bit saja atau urutannya terbalik, file tersebut bakal corrupt (rusak) dan tidak bisa dibuka sama sekali oleh sistem.

Konsekuensi Teknis UDP: Jika dipaksakan pakai UDP, besar kemungkinan file akan rusak karena adanya packet loss atau data yang tidak urut (out-of-order) di jaringan, dan UDP tidak punya mekanisme otomatis untuk memperbaiki itu.

2. Mekanisme Keandalan TCP (Teoritis Lanjutan)

Soal ini menguji pengetahuan tentang detail kerja TCP.

Soal: Dalam implementasi fitur Transfer File menggunakan TCP, jelaskan peran dari tiga mekanisme TCP berikut: Sequence Number, Acknowledgement Number (ACK), dan Window Size. Bagaimana ketiga mekanisme ini secara kolektif menjamin reliable data transfer dan flow control selama proses pengiriman file?

JAWAB :

Peran Mekanisme:
1. Sequence Number: Fungsinya seperti nomor halaman. Ini membantu penerima menyusun kembali potongan data biar urut seperti aslinya.
2. Acknowledgement Number (ACK): Ini adalah sinyal konfirmasi atau "tanda terima". Penerima memberitahu pengirim: "Paket nomor sekian sudah sampai, lanjut kirim berikutnya."
3. Window Size: Ini adalah rem-nya (flow control). Mengatur seberapa banyak data yang boleh dikirim sebelum menunggu konfirmasi, supaya penerima tidak kebanjiran data (buffer overflow).

Cara Kerjanya Bareng: Ketiganya kerja tim: Sequence Number memastikan urutan benar, ACK memicu pengiriman ulang kalau ada yang hilang, dan Window Size menjaga kecepatan kirim agar stabil sesuai kemampuan penerima.

3. Fungsi Header UDP (Pemahaman Detail)

Soal ini menguji pemahaman terhadap struktur dan peran header.

Soal: Sebutkan dan jelaskan fungsi dari empat (4) field yang terdapat dalam header UDP. Mengapa field Checksum seringkali diabaikan atau diatur ke nol pada implementasi UDP di IPv4, dan apa dampaknya terhadap keandalan data?

JAWAB: Fungsi Header UDP (Pemahaman Detail)

Empat Field Header UDP:

1. Source Port: Nomor port pengirim.
2. Destination Port: Nomor port penerima.
3. Length: Panjang total datagram.
4. Checksum: Nilai untuk mengecek error data.

Kenapa Checksum Sering 0? Di IPv4, checksum sering dimatikan (di-set 0) demi performa dan kecepatan (mengurangi latensi). Karena UDP mengejar kecepatan, pengecekan error dianggap membuang waktu. Risikonya, kalau ada data rusak di jalan, UDP tidak akan sadar dan data rusak itu tetap masuk ke aplikasi.

Kategori B: Perancangan dan Arsitektur (Desain)

4. Arsitektur Multi-Port Server (Desain Jaringan)

Soal ini menguji kemampuan merancang arsitektur server yang efektif.

Soal: Dalam proyek ini, Server Anda harus menyediakan dua layanan pada dua port yang berbeda (Port A untuk TCP dan Port B untuk UDP). Gambarkan skema arsitektur Server tersebut. Jelaskan tahapan proses Socket Programming pada Server untuk Port A (TCP Listening Socket) hingga siap menerima koneksi pertama, dan bedakan dengan inisialisasi pada Port B (UDP Datagram Socket).

JAWAB: Arsitektur Multi-Port Server

Skema Arsitektur: Server punya satu IP tapi buka dua "pintu".

a. Port A (TCP): Khusus buat Transfer File.
b. Port B (UDP): Khusus buat Notifikasi. Bedanya di Coding (Socket Programming):
a. Port A (TCP): Prosesnya lebih panjang. Server harus socket() → bind() → listen() (buka antrean) → terus accept() (nunggu koneksi). Baru bisa komunikasi.
b. Port B (UDP): Lebih simpel. Server cukup socket() → bind(). Setelah itu langsung siap terima paket (recvfrom) tanpa perlu listen atau accept.

5. Penanganan Konkurensi (Multi-Client)

Soal ini menguji solusi untuk menangani banyak klien secara bersamaan.

Soal: Untuk layanan Transfer File (TCP), Server harus mendukung multi-client (konkurensi). Jelaskan pendekatan yang Anda pilih (misalnya, Multi-Threading atau Multi-Processing). Mengapa pendekatan ini diperlukan pada TCP listening socket, dan bagaimana hal itu mencegah blocking ketika klien kedua mencoba terhubung saat klien pertama sedang melakukan transfer file?

JAWAB: Penanganan Konkurensi (Multi-Client)

Pendekatan: Pakai Multi-Threading.

Alasannya: Di TCP, fungsi accept() itu sifatnya blocking (menahan proses). Kalau tidak pakai multi-threading, saat server sibuk melayani transfer file Klien 1, Klien 2 yang mau masuk bakal dicuekin atau antre lama.

Mekanisme: Begitu accept() menerima koneksi baru, Main Thread (server utama) langsung membuat Worker Thread (petugas tambahan) untuk melayani klien tersebut. Jadi, server utama bisa langsung balik siaga menunggu klien berikutnya.

Kategori C: Implementasi dan Praktik Pemrograman (Praktikal)

6. Sintaks Socket (Implementasi Kode)

Soal ini menguji pengetahuan tentang API Socket dalam bahasa pemrograman tertentu.

Soal: Tuliskan pseudocode atau potongan kode dalam angka pemrograman pilihan Anda (misalnya Python/Java) yang menunjukkan angkah-langkah kritis pada sisi Client untuk:
1. Membuat TCP socket dan menghubungkannya (connect) ke Server.
2. Membuat UDP socket dan mengirimkan datagram pertama.

JAWAB: Sintaks Socket (Python)

TCP Client (Harus Connect): import socket # Bikin socket
TCP s = socket.socket(socket.AF_INET,
socket.SOCK_STREAM) # Wajib CONNECT dulu ke server
s.connect((‘192.168.1.10’, 8080))

UDP Client (Langsung Kirim): import socket # Bikin socket
UDP s = socket.socket(socket.AF_INET,
socket.SOCK_DGRAM) # Gak perlu connect, langsung
TEMBAK (sendto) msg = “Halo Server”.encode()
s.sendto(msg, (‘192.168.1.10’, 9090))

7. Validasi Integritas Data (Implementasi Keandalan)

Soal ini menguji mekanisme tambahan untuk memastikan kualitas layanan.

Soal: Walaupun TCP sudah andal, jelaskan satu (1) metode sederhana yang dapat Anda implementasikan pada sisi Client setelah menerima file dari Server untuk memverifikasi integritas data yang diterima (misalnya, untuk memastikan file tidak terpotong atau rusak). Tuliskan pseudocode untuk langkah validasi tersebut.

JAWAB :

Metode: Cek manual pakai Hash/Checksum Aplikasi (misalnya SHA-256).
Logika Pseudocode:

```
// 1. Terima file sampai selesai
file_content = read_file("file_diterima.dat")

// 2. Hitung hash file yang diterima hash_hitungan
= hitung_sha256(file_content)

// 3. Minta hash asli dari server hash_asli
= terima_dari_server()

// 4. Bandingkan
JIKA hash_hitungan == hash_asli MAKA
PRINT "File Aman (Valid)"
LAINNYA
PRINT "File Rusak (Corrupt)"
AKHIR JIKA
```

Kategori D: Analisis Kasus dan Debugging

8. Kasus Kegagalan UDP (Debugging/Troubleshooting)

Soal ini menguji pemahaman tentang kelemahan UDP.

Soal: Asumsikan Client mengirimkan 10 datagram notifikasi singkat berturut-turut ke Server menggunakan UDP. Tiba-tiba, jaringan mengalami packet loss yang tinggi. Jelaskan dua (2) skenario yang mungkin terjadi pada Server terkait 10 paket tersebut, dan apa yang harus dilakukan Application Layer pada Client jika notifikasi tersebut mutlak harus sampai ke Server?

JAWAB:

Dua Skenario di Server:

1. Data Bolong (Partial): Server cuma terima sebagian paket (misal paket 1, 3, 5), sisanya hilang di jalan.
2. Data Ngacak (Out-of-Order): Paket datangnya tidak urut (paket 10 sampai duluan sebelum paket 2), dan UDP memprosesnya apa adanya.

Solusi di Aplikasi Client: Karena UDP "masa bodoh", aplikasinya yang harus pintar. Kita perlu buat mekanisme Reliability Manual. Client harus menyematkan nomor urut dan

menunggu balasan "OK" (ACK) dari Server. Kalau dalam sekian detik tidak ada balasan, aplikasi Client harus otomatis kirim ulang datanya.

9. Port dan Socket dalam Konteks Server (Terminologi)

Soal ini menguji kejelasan terminologi teknis.

Soal: Dalam konteks Server yang berjalan:
1. Apa yang dimaksud dengan well-known port dan berikan satu contoh protokol/layanan yang menggunakannya?
2. Jelaskan perbedaan antara Port Number dan Socket Address (pasangan <IP Address>:<Port Number>), dan mengapa Socket Address adalah penentu unik sebuah koneksi.

JAWAB:

Well-known Port: Itu nomor port "spesial" (range 0–1023) yang sudah di-booking untuk layanan standar. Contohnya: Port 80 buat web (HTTP) atau 443 buat HTTPS.
Bedanya Port vs Socket Address:
• Port Number: Cuma nomor pintunya saja (misal: 80).
• Socket Address: Alamat lengkapnya, gabungan IP + Port (misal: 203.0.113.5:80).

• Uniknya: Socket Address jadi penentu unik karena dia menunjuk secara spesifik "Siapa" (IP) dan lewat "Pintu Mana" (Port) komunikasi itu terjadi di internet.

10. Fitur Alternatif (Kreativitas dan Justifikasi)

Soal ini menguji kemampuan berpikir kritis dan perancangan alternatif.

Soal: Jika Anda diminta untuk mengganti fitur Notifikasi Cepat (saat ini menggunakan UDP) dengan fitur Panggilan Suara Real-time (VoIP), apakah protokol yang digunakan (UDP) masih cocok? Jelaskan justifikasi Anda berdasarkan karakteristik protokol tersebut. Jika Anda memutuskan untuk mengubah protokol, jelaskan alasannya dan protokol apa yang akan Anda gunakan.

JAWAB:

Telepon (VoIP) itu butuh real-time. Dalam obrolan suara, "kecepatan" lebih penting daripada "kelengkapan". Kalau pakai TCP, pas sinyal jelek, suara bakal putus-putus parah (buffering) karena nunggu paket yang hilang dikirim ulang. Kalau pakai UDP, paling cuma ada suara "kresek" dikit (glitch) tapi obrolan jalan terus tanpa delay.